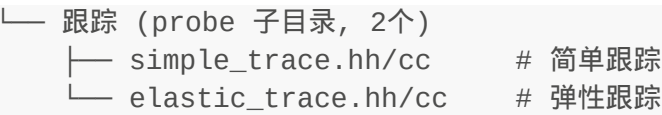


O3 CPU 代码结构分析

目录结构概览

/home/mikasa/0403/gem5/src/cpu/o3/	
├─ 核心文件 (16个)	
│ ├── cpu.hh/cc	# 主 CPU 类
│ ├── fetch.hh/cc	# 取指阶段
│ ├── bac.hh/cc	# 分支地址计算阶段
│ ├── ftq.hh/cc	# 取指目标队列
│ ├── decode.hh/cc	# 译码阶段
│ ├── rename.hh/cc	# 重命名阶段
│ ├── iew.hh/cc	# 发射/执行/写回阶段
│ ├── commit.hh/cc	# 提交阶段
│ ├── dyn_inst.hh/cc	# 动态指令实现
│ └─ limit.hh	# 全局常量定义
├─ 流水线后端支持 (8个)	
│ ├── inst_queue.hh/cc	# 指令队列 (IQ)
│ ├── lsq.hh/cc	# 加载存储队列顶层
│ ├── lsq_unit.hh/cc	# 每线程 LSQ
│ └─ rob.hh/cc	# 重排序缓冲 (ROB)
├─ 寄存器管理 (4个)	
│ ├── regfile.hh/cc	# 物理寄存器文件
│ ├── free_list.hh/cc	# 管理寄存器空闲列表
│ ├── rename_map.hh/cc	# 重命名映射
│ └─ scoreboard.hh/cc	# 记分板
├─ 内存依赖 (3个)	
│ ├── mem_dep_unit.hh/cc	# 内存依赖单元
│ ├── store_set.hh/cc	# Store Set 预测器
│ └─ dep_graph.hh	# 依赖图 (模板)
├─ 功能单元 (1个)	
│ └─ fu_pool.hh/cc	# 功能单元池
├─ 线程管理 (2个)	
│ ├── thread_context.hh/cc	# 线程上下文
│ └─ thread_state.hh/cc	# 线程状态
├─ 通信结构 (1个)	
│ └─ comm.hh	# 阶段间通信结构定义
├─ 指针指针 (1个)	
│ └─ dyn_inst_ptr.hh	# 动态指令指针定义
├─ 检查器 (2个)	
│ └─ checker.hh/cc	# 检查器支持



文件分类详解

一、核心流水线文件

1. cpu.hh / cpu.cc

类/结构	用途	关键成员
CPU	主 CPU 类，继承自 BaseCPU	- bac, ftq, fetch, decode, rename, iew, commit - regFile, freeList, renameMap, rob, scoreboard - timeBuffer, fetchQueue, decodeQueue, renameQueue, iewQueue - globalSeqNum, globalFTSeqNum - instList, removeList, activeThreads

关键函数:

- tick() - 主循环，依次调用各阶段 tick()
- drain() / drainResume() - 排空/恢复流水线
- getReg() / setReg() - 物理寄存器访问
- getArchReg() / setArchReg() - 架构寄存器访问（通过重命名映射）
- squashFromTC() - 从 ThreadContext 触发的 squash
- trap() / processInterrupts() - 异常/中断处理

2. fetch.hh / fetch.cc

类/结构	用途	关键成员
Fetch	取指阶段	- icachePort - I-cache 端口 - bac, ftq - 指向前端指针 - fetchBuffer[], fetchQueue[] - 取指缓冲区 - pc[], fetchOffset[] - PC 管理 - fetchPolicy - SMT 取指策略

关键函数:

- tick() - 取指阶段主循环
- fetch() - 核心取指逻辑
- fetchCacheLine() - 从 I-cache 取缓存行
- finishTranslation() - TLB 转换完成
- buildInst() - 创建动态指令
- getFetchingThread() - SMT 线程选择（轮询/IQ/LSQ/分支计数策略）

- `squashFromCommit()` / `squashFromDecode()` - 处理 squash

内部类:

- `IcachePort` - RequestPort，处理 I-cache 响应
- `FetchTranslation` - TLB 转换包装器
- `FinishTranslationEvent` - 延迟 TLB 错误处理

3. bac.hh / bac.cc

类/结构	用途	关键成员
BAC	分支地址计算阶段	- <code>bpu</code> - 分支预测器 - <code>ftq</code> - 取指目标队列 - <code>bacPC[]</code> - 解耦 PC（领先于 fetch） - <code>decoupledFrontEnd</code> - 是否启用解耦前端 - <code>fetchTargetWidth, maxFTPerCycle</code> - 解耦参数

关键函数:

- `tick()` - BAC 阶段主循环
- `generateFetchTargets()` - 解耦前端核心，生成取指目标
- `predict()` - 分支预测
- `updatePC()` - PC 更新
- `updatePreDecode()` - 预译码后更新分支预测
- `checkAndUpdateBPUSignals()` - 处理分支预测器信号
- `squashBpuHistories()` - Squash BPU 历史

解耦前端工作原理:

1. 使用 BTB 搜索指令流中的分支

2. 找到分支或达到搜索宽度时停止

3. 使用 BPU 预测分支方向

4. 创建取指目标（类似基本块），包含预测信息

5. 插入 FTQ，供 Fetch 消费

4. ftq.hh / ftq.cc

类/结构	用途	关键成员
FetchTarget	单个取指目标（类似基本块）	- <code>startPC, endPC</code> - 起始/结束地址 - <code>predTarg</code> - 预测目标地址 - <code>foundBranch, predTaken</code> - 分支信息 - <code>bpuHistory</code> - 分支预测历史

FTQ	取指目标队列	- <code>ftQueue[]</code> - 取指目标队列 - <code>headPtr[], tailPtr[]</code> - 队头/队尾指针
-----	--------	---

关键函数:

- `insert()` - 插入取指目标
- `readHead()` / `popHead()` - 读取/弹出队首
- `squash()` - 清除取指目标
- `finalize()` - 完成取指目标
- `isExitBranch()` / `inRange()` - 查询函数

5. decode.hh / decode.cc

类/结构	用途	关键成员
Decode	译码阶段	- <code>insts[]</code>, <code>skidBuffer[]</code> - 指令队列 - <code>fetchQueue</code>, <code>decodeQueue</code>, <code>renameQueue</code> - 时间缓冲 - <code>bdelayDoneSeqNum[]</code> - 分支延迟指令序列号

关键函数:

- `tick()` - 译码阶段主循环
- `decode()` - 核心译码逻辑
- `decodeInsts()` - 处理来自取指的指令，验证 PC 相对分支
- `squash()` - 处理 squash（分支预测错误检测）
- `sortInsts()` - 按线程分离指令

6. rename.hh / rename.cc

类/结构	用途	关键成员
Rename	重命名阶段	- <code>insts[]</code>, <code>skidBuffer[]</code> - 指令队列 - <code>historyBuffer[]</code> - 重命名历史缓冲 - <code>renameMap[]</code> - 重命名映射 - <code>freeList</code>, <code>scoreboard</code> - 资源管理 - <code>serializeInst[]</code> - 序列化指令

关键函数:

- `tick()` - 重命名阶段主循环
- `rename()` - 核心重命名逻辑
- `renameInsts()` - 执行寄存器重命名
- `renameSrcRegs()` / `renameDestRegs()` - 源/目的寄存器重命名
- `handleMiscRegWaw()` - 处理杂项寄存器写依赖
- `serializeAfter()` - 序列化处理
- `squash()` / `doSquash()` / `removeFromHistory()` - Squash 处理

内部结构:

- `RenameHistory` - 存储重命名历史（序列号、架构寄存器、新旧物理寄存器）

7. iew.hh / iew.cc

类/结构	用途	关键成员
IEW	发射/执行/写回阶段	- instQueue - 指令队列 - ldestQueue - 加载存储队列 - scoreboard, fuPools[] - 资源 - insts[], skidBuffer[] - 指令队列
关键函数: • tick() - IEW 阶段主循环 • dispatch() / dispatchInsts() - 分发指令到 IQ/LSQ • executeInsts() - 执行指令 • writebackInsts() - 写回结果，唤醒依赖指令 • wakeDependents() - 唤醒依赖指令 • rescheduleMemInst() / replayMemInst() - 内存指令重放 • squash() / blockMemInst() / retryMemInst() - 内存指令处理 • checkMisprediction() - 分支预测错误检查		

8. commit.hh / commit.cc

类/结构	用途	关键成员
Commit	提交阶段	- rob - 重排序缓冲 - renameMap[] - 重命名映射 - pc[] - 各线程 PC 状态 - youngestSeqNum[], lastCommittedSeqNum[] - 序列号管理 - squashAfterInst[] - SquashAfter 支持 - commitPolicy - SMT 提交策略
关键函数: • tick() - 提交阶段主循环 • commit() - 提交主逻辑 • commitInsts() - 提交指令 • commitHead() - 提交 ROB 头部指令 • getInsts() - 从重命名获取指令到 ROB • markCompletedInsts() - 标记已完成指令 • squashFromTrap() / squashFromTC() / squashFromSquashAfter() - Squash 处理 • handleInterrupt() - 中断处理 • getCommittingThread() - SMT 线程选择（轮询/最旧优先）		

二、流水线后端支持文件

9. inst_queue.hh / inst_queue.cc

类/结构	用途	关键成员
InstructionQueue	指令队列	- instList[] - 指令列表 - instsToExecute - 就绪指令列表 - readyInsts[] - 就绪指令优先级队列 - nonSpecInsts - 非推测指令映射 - dependGraph - 依赖图 - memDepUnit[] - 内存依赖单元
关键函数:		
• insert() / insertNonSpec() / insertBarrier() - 插入指令 • scheduleReadyInsts() - 调度就绪指令 • getInstToExecute() - 获取要执行的指令 • wakeDependents() - 唤醒依赖指令 • rescheduleMemInst() / replayMemInst() - 内存指令处理 • deferMemInst() / blockMemInst() / retryMemInst() - 延迟/阻塞/重试内存指令 • violation() - 内存顺序违规 • squash() / doSquash() - Squash 处理		
内部类:		
• FUCompletion - 功能单元完成事件 • PqCompare - 优先级队列比较器（按序列号排序） • IQUnit - 每线程 IQ 单元（SMT 管理） • ListOrderEntry - 年龄顺序列表条目		

10. lsq.hh / lsq.cc

类/结构	用途	关键成员
LSQ	LSQ 顶层	- dcachePort - D-cache 端口 - thread[] - 每线程 LSQ 单元 - recvRespThrottling - 响应节流 - lsqPolicy - SMT 策略

关键函数:		
• tick() - LSQ 主循环 • insertLoad() / insertStore() - 插入加载/存储 • executeLoad() / executeStore() - 执行加载/存储 • commitLoads() / commitStores() - 提交加载/存储 • writebackStores() - 存储写回内存 • squash() - Squash 处理 • violation() - 检测内存顺序违规 • recvTimingResp() - 处理内存响应 • pushRequest() - 推送内存请求		

内部类:

- `DcachePort` - D-cache 端口，处理内存请求/响应/侦听
- `LSQRequest` - 内存请求基类
- `SingleDataRequest` - 单一数据请求
- `SplitDataRequest` - 分割数据请求
- `UnsquashableDirectRequest` - 不可 squash 的直接请求

11. `lsq_unit.hh / lsq_unit.cc`

类/结构	用途	关键成员
<code>LSQUnit</code>	每线程 LSQ 单元	- <code>loadQueue, storeQueue</code> - 加载/存储队列 - <code>loadOffset, storeOffset</code> - 队列偏移 - <code>memDepEntry</code> - 内存依赖条目 - <code>memDepViolator</code> - 内存依赖违规者

关键函数:

- `executeLoad() / executeStore()` - 执行加载/存储
- `checkViolations()` - 检查内存顺序违规
- `commitLoad() / commitLoads() / commitStores()` - 提交操作
- `writebackStores()` - 存储写回
- `completeDataAccess()` - 完成数据访问
- `recvTimingResp()` - 处理内存响应

内部结构:

- `LSQEntry / LQEntry / SQEntry` - 队列条目
- `WritebackEvent` - 存储写回事件

12. `rob.hh / rob.cc`

类/结构	用途	关键成员
<code>ROB</code>	重排序缓冲	- <code>instList[]</code> - 指令列表 - <code>head, tail</code> - 头部/尾部迭代器 - <code>squashIt[]</code> - Squash 迭代器 - <code>robPolicy</code> - SMT 策略

关键函数:

- `insertInst()` - 插入指令到 ROB
- `readHeadInst() / retireHead()` - 读取/移除头部指令
- `squash() / doSquash()` - Squash 处理
- `isHeadReady() / canCommit()` - 提交就绪检查
- `updateHead() / updateTail()` - 更新头/尾指针
- `countInsts()` - 统计指令数

三、寄存器管理文件

13. regfile.hh / regfile.cc

类/结构	用途	关键成员
PhysRegFile	物理寄存器文件	- regs[] - 各类寄存器数组 - regClasses - 寄存器类列表

关键函数:

- getReg() / setReg() - 物理寄存器读写（多个重载）
- initRegs() - 初始化寄存器
- totalNumPhysRegs() - 总物理寄存器数

支持的寄存器类型:

- IntRegClass - 整数寄存器
- FloatRegClass - 浮点寄存器
- VecRegClass - 向量寄存器
- VecPredRegClass - 向量谓词寄存器
- MatRegClass - 矩阵寄存器
- CCRRegClass - 条件码寄存器

14. free_list.hh / free_list.cc

类/结构	用途	关键成员
UnifiedFreeList	统一空闲列表	- freeLists[] - 每类空闲列表
SimpleFreeList	单类空闲列表	- freeList - 空闲寄存器列表

关键函数:

- getReg() - 分配空闲物理寄存器
- addReg() / addRegs() - 返还物理寄存器
- numFreeRegs() - 查询空闲寄存器数
- reset() - 重置空闲列表

15. rename_map.hh / rename_map.cc

类/结构	用途	关键成员
UnifiedRenameMap	统一重命名映射	- maps[] - 每类映射
SimpleRenameMap	单类重命名映射	- map - 映射表

关键函数:

- `rename()` - 分配新物理寄存器并更新映射
- `lookup()` - 查找架构寄存器的物理映射
- `setEntry()` - 设置映射
- `minFreeEntries()` - 最小空闲条目数

16. scoreboard.hh / scoreboard.cc

类/结构	用途	关键成员
<code>Scoreboard</code>	记分板	- <code>regScoreboard</code> - 寄存器就绪位图（位向量）

关键函数:

- `getReg()` - 检查寄存器是否就绪
- `setReg()` - 设置寄存器为就绪
- `unsetReg()` - 清除寄存器就绪标志

实现原理: 使用位向量存储就绪状态，每个字节存储 8 个寄存器的状态，以节省空间。

四、内存依赖文件

17. mem_dep_unit.hh / mem_dep_unit.cc

类/结构	用途	关键成员
<code>MemDepUnit</code>	内存依赖单元	- <code>memDepList</code> - 内存依赖列表 - <code>memInsts</code> - 内存指令映射 - <code>storeSet</code> - Store set 预测器

关键函数:

- `insert() / insertNonSpec() / insertBarrier()` - 插入内存指令
- `completeInst() / nonSpecInstReady()` - 标记指令完成
- `violation()` - 记录内存顺序违规
- `reschedule() / replay()` - 重放/回放内存指令
- `squash()` - Squash 处理
- `wakeDependents()` - 唤醒依赖指令

内部结构:

- `MemDepEntry` - 内存依赖条目

18. store_set.hh / store_set.cc

类/结构	用途	关键成员
<code>StoreSet</code>	Store set 预测器	- <code>SSIT</code> - Store Set ID Table - <code>LFST</code> - Last Fetched Store Table

关键函数:

- `violation()` - 记录违规，学习依赖
- `insertLoad()` / `insertStore()` - 插入加载/存储
- `checkInst()` - 检查预测依赖
- `issued()` - 记录指令发射
- `squash()` - Squash 清除

内部结构:

- `SSITEntry` - SSIT 缓存条目
- `LFSTEntry` - LFST 缓存条目

预测原理:

- SSIT: 将 PC 映射到 store set ID
- LFST: 记录每个 set 的最后一条存储
- 预测加载依赖最近同一 set 中的存储

19. dep_graph.hh

类/结构	用途	关键成员
<code>DependencyGraph<T></code>	依赖图模板类	- <code>dependGraph</code> - 依赖图数组

关键函数:

- `insert()` - 插入指令到依赖图
- `setInst()` / `clearInst()` - 设置/清除生产者
- `remove()` - 移除指令
- `pop()` - 弹出最新依赖
- `setInst()` / `clearInst()` - 设置/清除指令

内部结构:

- `DependencyEntry` - 依赖链节点

数据结构: 使用链表实现寄存器依赖关系，每个物理寄存器维护一个生产者-消费者链。

五、功能单元文件

20. fu_pool.hh / fu_pool.cc

类/结构	用途	关键成员
------	----	------

类/结构	用途	关键成员
FUPool	功能单元池	- fuIndices - 功能单元索引
		- busy - 忙闲状态
		- opLatencies - 操作延迟
		- opClasses - 操作类型
		- fuQueues - 功能单元队列

关键函数:

- getUnit() - 获取空闲功能单元
- freeUnitNextCycle() - 调度下一周期释放功能单元
- processFreeUnits() - 处理释放
- isCapable() - 检查能力

内部结构:

- FUIdxQueue - 功能单元索引循环队列
- FUDesc - 功能单元描述

六、线程管理文件

21. thread_context.hh / thread_context.cc

类/结构	用途	关键成员
ThreadContext	线程上下文	- cpu - CPU 指针
		- thread - 线程状态
		- 实现 gem5::ThreadContext 接口

关键函数:

- 架构寄存器访问：readIntReg(), readFloatReg(), setIntReg() 等
- PC 访问：pcState(), pcState()
- 线程控制：activate(), suspend(), halt()
- 事件队列：pcEventQueue(), comInstEventQueue()

22. thread_state.hh / thread_state.cc

类/结构	用途	关键成员
ThreadState	线程状态	- _status - 线程状态
		- process - 进程指针
		- pcEventQueue, comInstEventQueue - 事件队列
		- trapPending, noSquashFromTC - 状态标志

关键函数:

- activate() / suspend() / halt() - 线程控制

- `status()` - 状态查询
- `trap()` - 陷阱处理

七、通信文件

23. comm.hh

通信结构定义文件，包含所有阶段间通信的数据结构：

结构体	用途	关键成员
FetchStruct	取指->译码	<code>insts[]</code> , <code>fetchFault</code> , <code>fetchFaultSN</code>
DecodeStruct	译码->重命名	<code>insts[]</code>
RenameStruct	重命名->IEW	<code>insts[]</code>
IEWStruct	IEW->提交	<code>insts[]</code> , <code>mispredictInst[]</code> , <code>squashedSeqNum[]</code> , <code>pc[]</code> , <code>squash[]</code> , <code>branchTaken[]</code>
IssueStruct	发射->执行	<code>insts[]</code>
TimeStruct	反向通信	包含所有阶段的通信结构 <code>fetchInfo</code> , <code>decodeInfo</code> , <code>renameInfo</code> , <code>iewInfo</code> , <code>commitInfo</code> <code>decodeBlock</code> , <code>renameBlock</code> 等 stall 信号

八、辅助文件

24. dyn_inst_ptr.hh

动态指令指针定义文件：

```
class DynInst;

using DynInstPtr = RefCountingPtr<DynInst>;
using DynInstConstPtr = RefCountingPtr<const DynInst>;
```

25. dyn_inst.hh / dyn_inst.cc

类描述	关键特性
继承	<code>ExecContext</code> , <code>RefCounted</code>
状态管理	32 位状态标志，跟踪指令在流水线中的位置

类描述	关键特性
寄存器映射	源/目的寄存器的架构和物理映射
执行结果	<code>_result</code> (整数/浮点)
跟踪数据	<code>Trace::InstRecord</code>
微操作支持	<code>_macroop</code> , <code>isMicroop()</code> , <code>isFirstMicroop()</code> 等
内存操作	<code>isLoad()</code> , <code>isStore()</code> , <code>isAtomic()</code> 等
控制指令	<code>isControl()</code> , <code>predTaken()</code> 等
序列化	<code>isSerializeBefore()</code> , <code>isSerializeAfter()</code> 等

关键函数:

- `execute()` - 执行指令
- `initiateAcc()` / `completeAcc()` - 内存访问
- `renameSrcReg()` / `renameDestReg()` - 寄存器重命名
- `getRegOperand()` / `setRegOperand()` - 执行时寄存器访问
- `readMiscReg()` / `setMiscReg()` - 杂项寄存器访问

状态查询函数:

- `readyToIssue()`, `isIssued()`, `isExecuted()`
- `readyToCommit()`, `isCommitted()`
- `isSquashed()`, `isInIQ()`, `isInLSQ()`, `isInROB()`

26. limits.hh

全局常量定义:

```
static constexpr int MaxWidth = 16;           // 最大流水线宽度
static constexpr int MaxThreads = 4;           // 最大 SMT 线程数
```

27. checker.hh / checker.cc

检查器支持，用于动态验证指令执行结果。

九、跟踪文件

28. probe/simple_trace.hh / probe/simple_trace.cc

简单跟踪器，跟踪指令取指和提交。

类/结构	用途
------	----

类/结构	用途
SimpleTrace	继承 ProbeListenerObject，实现简单指令跟踪
关键函数:	
- traceFetch() - 跟踪取指 - traceCommit() - 跟踪提交	

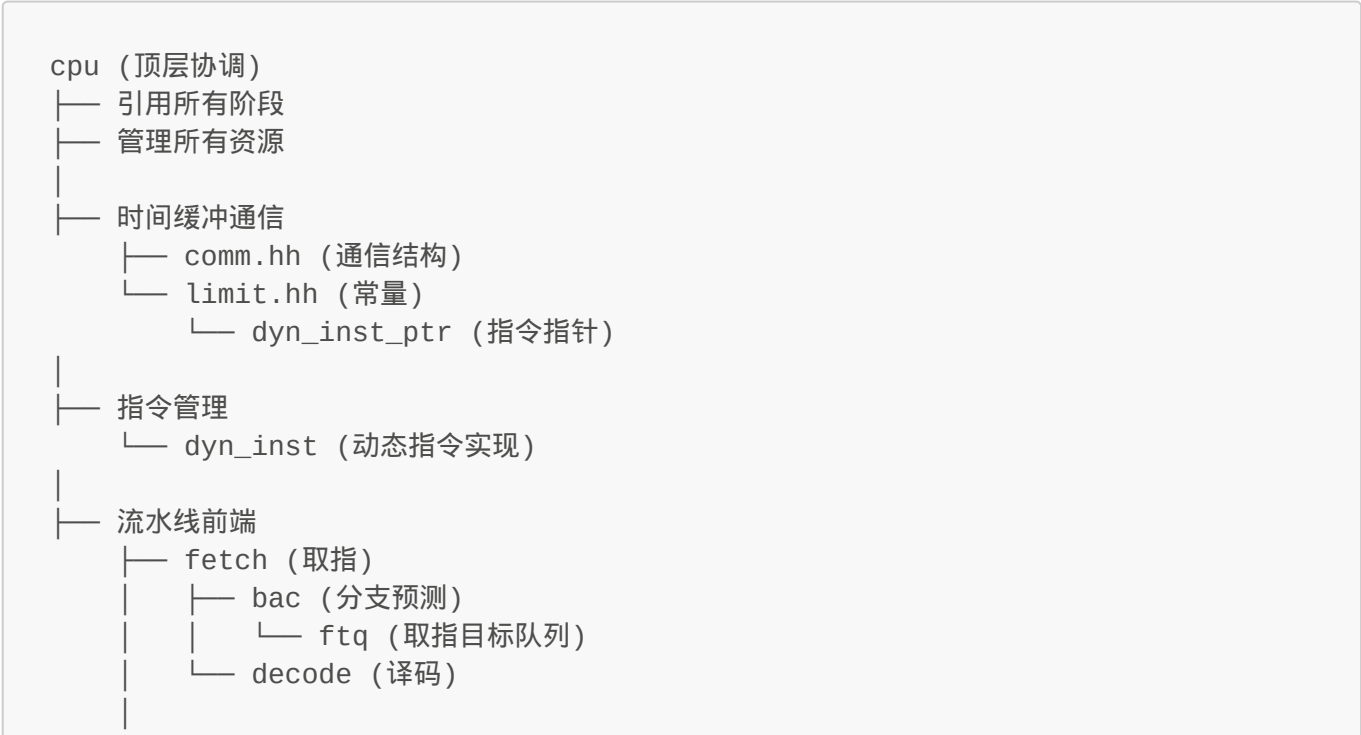
29. probe/elastic_trace.hh / probe/elastic_trace.cc

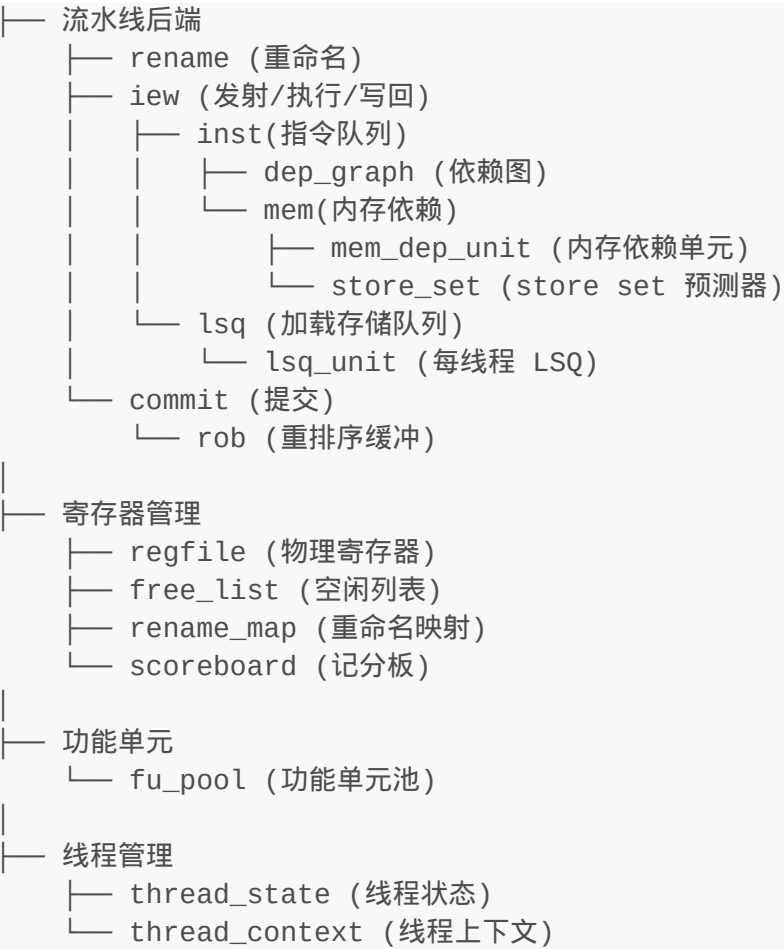
弹性跟踪器，生成详细依赖跟踪用于 TraceCPU 回放。

类/结构	用途
ElasticTrace	继承 ProbeListenerObject，实现详细依赖跟踪
关键函数:	
- 记录指令时序和依赖 - 计算计算延迟 - 生成 protobuf 跟踪	
内部结构:	
- InstExecInfo - 临时执行信息存储 - TraceInfo - 最终跟踪记录	

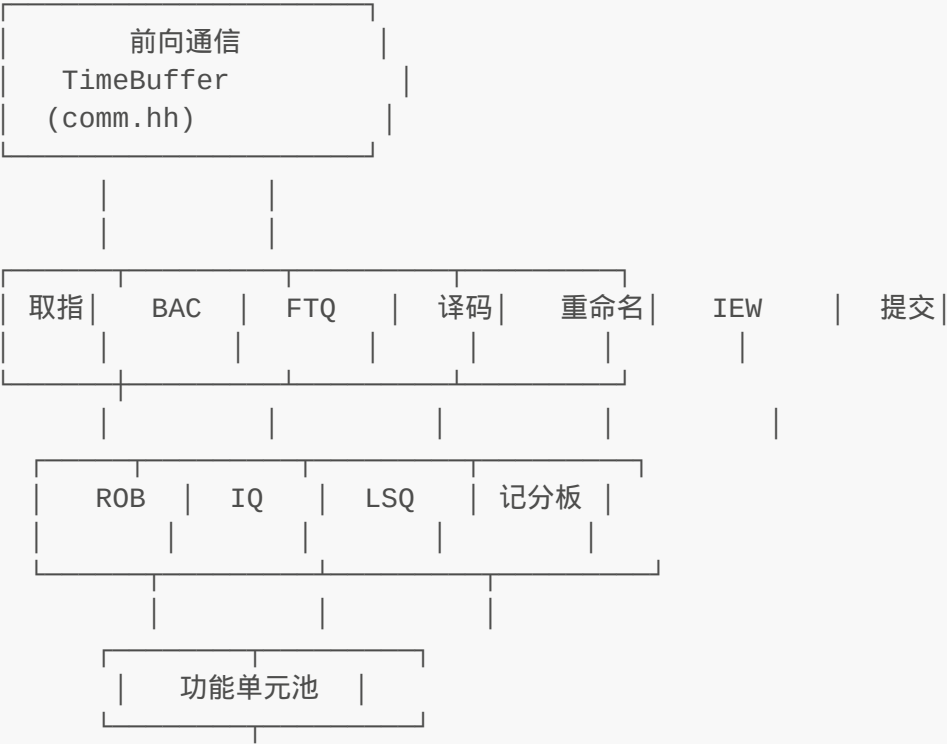
代码依赖关系图

核心依赖关系





阶段间数据流



关键设计模式

1. 时间缓冲区模式

使用 **TimeBuffer** 实现阶段间通信：

- **正向队列**：指令向前传播
- **反向通信**：Squash、stall、空闲条目数等信号向后传播
- **延迟控制**：通过队列深度模拟流水线延迟

2. SMT 策略模式

线程资源分配策略：

- **Dynamic**: 动态分配
- **Partitioned**: 固定分区
- **Threshold**: 阈值限制

取指策略：

- RoundRobin: 轮询取指
- IQCount: 按 IQ 占用率取指
- LSQCount: 按 LSQ 占用率取指
- BranchCount: 按分支数取指

提交策略：

- RoundRobin: 轮询提交
- OldestFirst: 最旧指令优先

3. 内存依赖预测

使用 Store Set 预测器预测 load-store 依赖：

- **SSIT**: Store Set ID Table，将 PC 映射到 set ID
- **LFST**: Last Fetched Store Table，记录每个 set 的最后存储
- **学习机制**: 检测到违规时学习正确的依赖关系

4. 解耦前端

BAC 和 Fetch 分离运行：

- **BAC** 独立生成取指目标（使用 BTB 搜索）
- **FTQ** 缓冲取指目标
- **Fetch** 消费取指目标，解耦关键路径

编译时配置

主要配置参数通过 **BaseO3CPUParams** 传入：

流水线参数：

- `fetchWidth, decodeWidth, renameWidth, commitWidth, issueWidth, wbWidth`
- `fetchBufferSize, decodeToFetchDelay, renameToDecodeDelay` 等

资源大小：

- `numPhysIntRegs, numPhysFloatRegs, numPhysVecRegs` 等
- `robSize, iqSize, lqSize, sqSize`

分支预测：

- `branchPred` - 分支预测器配置
- `decoupledFrontEnd` - 是否启用解耦前端
- `fetchTargetWidth, maxFTPPerCycle` - 解耦参数

SMT 配置：

- `numThreads` - 线程数
- `fetchPolicy, commitPolicy` - 策略
- `smtPolicy` (IQ, LSQ, ROB) - 资源分配策略

总结

O3 CPU 实现了完整的乱序超标量处理器模型，包含：

1. **9 个流水线阶段**：BAC, Fetch, Decode, Rename, IEW, Commit + 辅助结构
2. **完整的寄存器重命名**：支持多类寄存器、重命名映射、记分板
3. **乱序执行机制**：依赖图、功能单元调度、精确异常
4. **内存子系统**：LSQ、内存依赖预测、顺序检查
5. **SMT 支持**：多线程同时执行、多种资源分配策略
6. **解耦前端**：BAC/FTQ 分离分支预测和取指
7. **详细的跟踪和检查**：支持多种跟踪模式、检查器验证